

Imports

```
1 # @title
2 import torch
3 import math
4
5 from tqdm.auto import tqdm
6 from sklearn.cluster import KMeans
7 from sklearn.preprocessing import StandardScaler
```

```
1 # @title
2 class SpectralLayout:
3     """
4     Single source of truth for all tensor dimensions, slicing structures,
5     and physics routing to eliminate magic numbers across the architecture.
6     """
7     # --- 0. Core Physical Constants ---
8     EPS_MATH = 1e-8      # Algebraic safety (divide-by-zero prevention)
9     EPS_PHYSICS = 1e-6   # Minimum threshold for spatial/mass existence
10
11     # --- 1. Core Topological Truths ---
12     COEFF_DIM_KEYS = ["Segments", "Bounds", "Chebyshev"]
13     META_DIM_KEYS = ["Channels"]
14
15     PHYSICS_DIMS_COEFFS = len(COEFF_DIM_KEYS)
16     PHYSICS_DIMS_META = len(META_DIM_KEYS)
17
```

```
1 # @title
2 class _GlobalCanvasFramer:
3     """Pads the entire ND tensor to a perfect global power of 2."""
4     def pad_to_pow2(self, raw_nd_data):
5         shape = raw_nd_data.shape
6         pow2_shape = [1 << (s - 1).bit_length() if s > 1 else 1 for s in shape]
```

```
1 # @title
2 class _BaseLeafMinter:
3     """Projects raw 1D data into base Level 0 single-curve polynomials."""
4     def __init__(self, leaf_basis, recon_basis):
5         self.leaf_basis = leaf_basis
6         self.recon_basis = recon_basis
7
8     def mint(self, raw_1d_data, spatial_intervals):
9         patches = self._chunk(raw_1d_data, spatial_intervals)
10        coeffs_mu = self._project(patches)
11        coeffs_dual = coeffs_mu.unsqueeze(1).expand(-1, 2, -1)
12        sigma, max_err = self._calculate_error(patches, coeffs_mu)
13
14        return {
15            "coeffs": coeffs_dual,
16            "intervals": spatial_intervals,
17            "sigma": sigma,
18            "max_err": max_err
19        }
```

```
1 # @title
2 class _EvaluationEngine:
3     """Manages dynamic Chebyshev bases and executes discrete polynomial evaluation."""
4     def __init__(self):
5         self.cache = {}
6
7     def evaluate(self, coeffs_list, lengths):
8         """Evaluates a list of [4] coefficients into variable-length discrete segments."""
9         segments = []
```

```
1 # @title
2 class Interface:""
```

```
1 # @title
2 def fast_pytorch_kmeans(x, num_clusters, max_iters=50):
```

▼ Z-Spectral Compressor

▼ compressor

```
1 # @title
2 class TopologySerializer:
```

```
1 # @title
2 class ZSpectralCompressor:
```

▼ compressor validation

```
1 # @title
2 import torch
3 import urllib.request
4 from PIL import Image
5 import torchvision.transforms.functional as TF
6 import matplotlib.pyplot as plt
7
8 class VisionGeometryConfig:
```

```
1 # @title
2 def test_ultimate_quadtree_compression(merge_threshold=0.05, k_clusters=256):
3     print("Fetching High-Res 1024x1024 Image (1.05 MB)...")
```

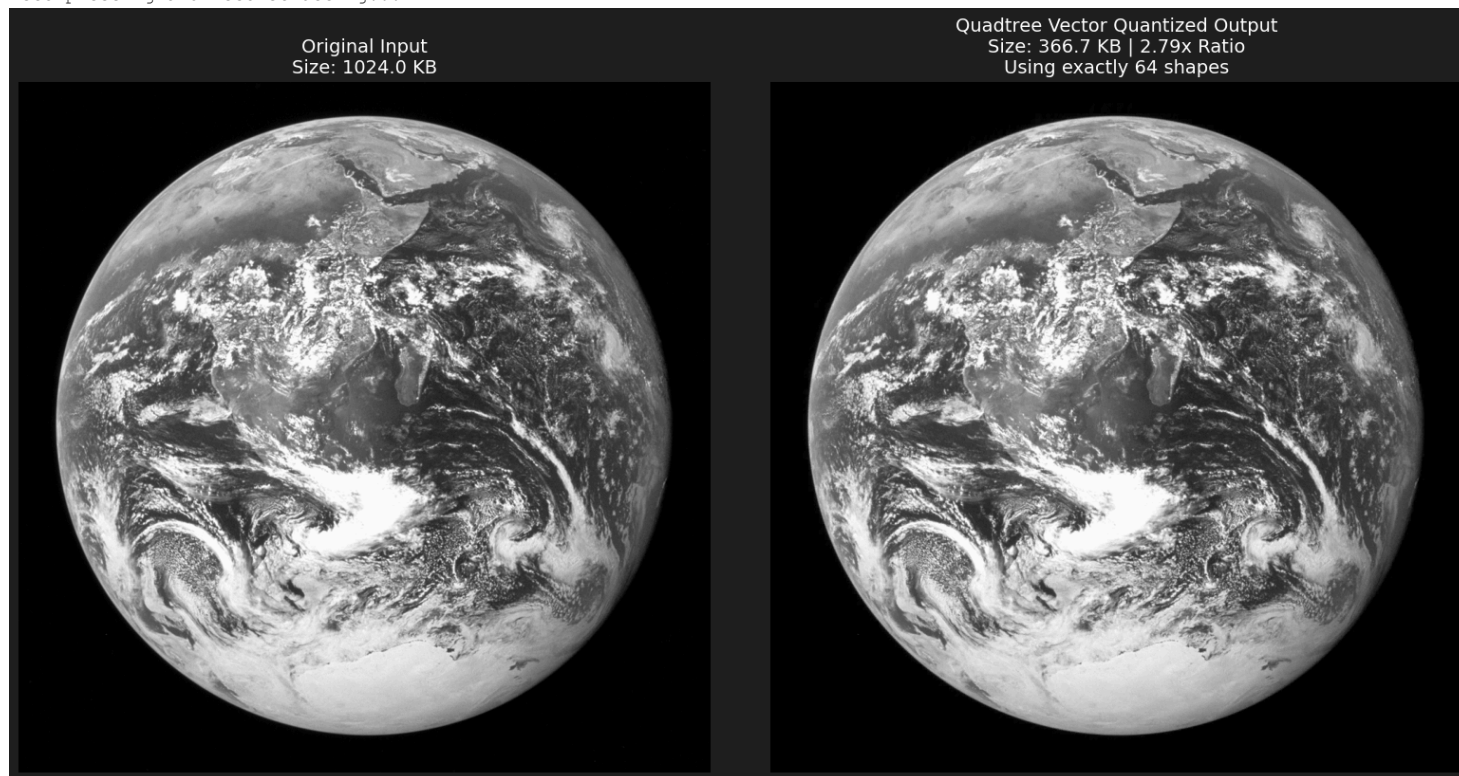
```
1 # @title
2 import matplotlib.pyplot as plt
3
4 def visualize_ultimate_compression(merge_threshold=0.05, k_clusters=256):
5     print("Fetching High-Res 1024x1024 Image...")
```

```
1 # @title
2 _merge_threshold = 0.25
3 _k_clusters = 64
4
5 test_ultimate_quadtree_compression("
9
10 visualize_ultimate_compression("
```

```
Fetching High-Res 1024x1024 Image (1.05 MB)...  
Compressing and Serializing Topology...
```

```
--- ULTIMATE QUADTREE SCALING (1024x1024) ---  
Active Tokens:          166550  
Uncompressed Image:     1024.00 KB  
Total Compressed Size:   366.71 KB  
-> Codebook Shapes:      0.75 KB  
-> Translation (c0):     162.65 KB  
-> Shape Labels:         162.65 KB  
-> Binary Tree Mask:     40.66 KB <-- The Magic!
```

```
True Byte Ratio:        2.79x  
Fetching High-Res 1024x1024 Image...  
Compressing and Serializing...  
Decompressing and Reconstructing...
```



compression optimizer

```
1 # @title  
2 class RateDistortionOptimizer:""
```

compression optimizer validation

Using Hardware: CUDA

```
--- JWST FULL COLOR GPU COMPRESSION ---  
--- GPU Pareto Sweep (Finding the Goldilocks Zone) ---  
Thresh 0.1: Tokens 262144 | MSE 0.000000 | Score: 2.7259  
Thresh 0.2: Tokens 262143 | MSE 0.000000 | Score: 1.4534  
Thresh 0.5: Tokens 235625 | MSE 0.000022 | Score: 0.8664  
Thresh 0.8: Tokens 182496 | MSE 0.000101 | Score: 0.7598  
Thresh 1.1: Tokens 127300 | MSE 0.000275 | Score: 0.6976  
Thresh 1.5: Tokens 78113 | MSE 0.000644 | Score: 0.6523  
Thresh 2.0: Tokens 21878 | MSE 0.001498 | Score: 0.6508
```

```
1 # @title  
2 import torch
```

```

3 import torchvision.transforms.functional as TF
4 from PIL import Image
5
6 # 1. DEFINE DEVICE FIRST
7 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8 print(f"Using Hardware: {device.type.upper()}")
9
10 def move_model_to_device(interface, device):""
11
12 # 2. INITIALIZE AND MOVE INTERFACE
13 config = VisionGeometryConfig()
14 interface = Interface(config)
15 interface = move_model_to_device(interface, device) # Now 'device' is defined!
16
17 optimizer = RateDistortionOptimizer(interface)
18
19 # --- THE MISSING BLOCK ---
20 print(f"Loading JWST image to GPU...")
21 color_img = Image.open("jwst_image.jpg").convert('RGB')
22 full_color_tensor = TF.to_tensor(color_img).to(device)
23 c, h, w = full_color_tensor.shape
24 full_metadata = {"shape": (h, w)}
25
26 # Hardcode the winning parameters from our earlier Pareto Sweep!
27 optimal_thresh = 1.1
28 optimal_k = 64
29 # -----
30
31 # 3. Final Tiled Run with Metric Tracking
32 config.merge_threshold = optimal_thresh
33 final_interface = Interface(config)
34 final_interface = move_model_to_device(final_interface, device) # Keep sync'd
35 final_compressor = ZSpectralCompressor(final_interface, k_clusters=optimal_k)
36
37 color_canvas = torch.zeros_like(full_color_tensor, device=device)
38 tile_size = 1024
39 total_raw_bytes, total_compressed_bytes, total_tokens = 0, 0, 0
40 codebook_paid = [False, False, False]
41
42 # Calculate total tiles for the progress bar
43 total_tiles = math.ceil(h / tile_size) * math.ceil(w / tile_size)
44
45 for i in range(3):""
46
47 print(f"\n--- FINAL RESULTS ---")
48 print(f"Compression Ratio: {total_raw_bytes / total_compressed_bytes:.2f}x")
49 print(f"Final Size: {total_compressed_bytes / (1024*1024):.2f} MB")
50
51 # Final result back to CPU for saving
52 TF.to_pil_image(color_canvas.cpu()).save("jwst_final_goldilocks.jpg", quality=100)

```

Using Hardware: CUDA
Loading JWST image to GPU...

Processing Channel 1/3...

Compressing Tiles: 100% 36/36 [02:09<00:00, 1.34tile/s]

Processing Channel 2/3...

Compressing Tiles: 100% 36/36 [01:50<00:00, 1.98tile/s]

Processing Channel 3/3...

Compressing Tiles: 100% 36/36 [01:56<00:00, 1.70tile/s]

--- FINAL RESULTS ---

Compression Ratio: 4.17x
Final Size: 20.13 MB

```

1 # @title
2 import matplotlib.pyplot as plt
3 from PIL import Image
4
5 # Load the saved images from the virtual file system
6 original_img = Image.open("jwst_image.jpg")
7 reconstructed_img = Image.open("jwst_final_goldilocks.jpg")
8
9 # Set up a massive side-by-side plot

```

```
10 fig, axes = plt.subplots(1, 2, figsize=(24, 12))
11
12 # Plot Original
13 axes[0].imshow(original_img)
14 axes[0].set_title("Original Uncompressed (84.00 MB)", fontsize=20, pad=15)
15 axes[0].axis('off')
16
17 # Plot Reconstructed
18 axes[1].imshow(reconstructed_img)
19 axes[1].set_title("Continuous Geometry Recon (20.13 MB | 4.17x Ratio)", fontsize=20, pad=15)
20 axes[1].axis('off')
21
22 # Render
23 plt.tight_layout()
24 plt.show()
```

Original Uncompressed (84.00 MB)



Continuous Geometry Recon (20.13 MB | 4.17x Ratio)

